

EXPERIMENT 8

Student Name: Vinay Tiwari

Branch: AIT-CSE

Semester: 4

Subject Name: Database Management System

UID: 24BAI70919

Section/Group: 24AIT_KRG-G2

Subject Code: 24CSH-298

Experiment 8 – Design of a Parameterized PL/SQL Stored Procedure for Employee Count by Gender (CData, Toddle)

Experiment

Experiment 8: Designing and implementing a parameterized PL/SQL stored procedure that accepts gender as an input parameter and returns the total number of employees based on the given gender. This experiment demonstrates procedural programming, parameterized logic, and dynamic data retrieval in a database environment.

Aim

To design and implement a stored procedure that accepts gender as an input parameter and returns the total number of employees based on the gender passed, thereby demonstrating procedural programming and parameterized logic in PL/SQL.

Objective

- To understand the structure and components of a PL/SQL stored procedure.
- To implement IN and OUT parameters in stored procedures.
- To pass dynamic input values to procedures.
- To apply aggregate functions for data analysis.
- To develop reusable database logic for real-world business scenarios.

Software Requirements

Database Management System:

- Oracle Database Express Edition (Oracle XE)
- PostgreSQL Database

Database Administration Tool / Client Tool:

EXPERIMENT 8

- Oracle SQL Developer (for Oracle XE)
- pgAdmin (for PostgreSQL)

Problem Statement

Organizations often require flexible reporting mechanisms to analyze workforce distribution based on gender for compliance and internal analytics. A dynamic and reusable solution is needed to efficiently retrieve employee counts based on gender input.

Practical / Experiment Steps

- Create an employee table with relevant attributes.
- Insert sample data into the employee table.
- Design a stored procedure that accepts gender as an input parameter.
- Use an aggregate function to count employees based on the given gender.
- Return the result using an OUT parameter.
- Execute the stored procedure with different gender inputs.
- Analyze the output.

Procedure

1. Open Oracle SQL Developer or pgAdmin and connect to the database.
2. Create the employee table with fields such as id, name, and gender.
3. Insert sample records into the table.
4. Create a PL/SQL stored procedure with IN and OUT parameters.
5. Write a SELECT COUNT(*) query inside the procedure.
6. Compile the procedure successfully.
7. Execute the procedure using different gender values.
8. Display the output.
9. Capture results for documentation.

Input / Output Details

Input

- Table:

EXPERIMENT 8

- employee (id, name, gender)
- Input Parameter:
 - Gender value (e.g., 'Male', 'Female')

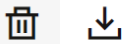
Step-wise Output

Step 1 – Create Employee Table

```
1  -- Step 1: Create Employee Table START
2  CREATE TABLE employee (
3      id NUMBER PRIMARY KEY,
4      name VARCHAR2(50),
5      gender VARCHAR2(10)
6  );
7  -- Step 1: Create Employee Table END
8
9
10 -- Step 2: Insert Sample Data START
```



Query result **Script output** DBMS output Explain Plan SQL history



```
SQL> CREATE TABLE employee (
      id NUMBER PRIMARY KEY,
      name VARCHAR2(50),
      gender VARCHAR2(10)...
```

[Show more...](#)

Table EMPLOYEE created.

Elapsed: 00:00:00.026

EXPERIMENT 8

Step 2 – Insert Sample Data

```

9
10 -- Step 2: Insert Sample Data START
11 INSERT INTO employee VALUES (1, 'Amit', 'Male');
12 INSERT INTO employee VALUES (2, 'Neha', 'Female');
13 INSERT INTO employee VALUES (3, 'Raj', 'Male');
14 INSERT INTO employee VALUES (4, 'Priya', 'Female');
15 INSERT INTO employee VALUES (5, 'Karan', 'Male');
16 COMMIT;
17 -- Step 2: Insert Sample Data END

```

Query result

Script output

DBMS output

Explain Plan

SQL history



1 row inserted.

Elapsed: 00:00:00.002

SQL> INSERT INTO employee VALUES (5, 'Karan', 'Male')

1 row inserted.

Elapsed: 00:00:00.002

SQL> COMMIT

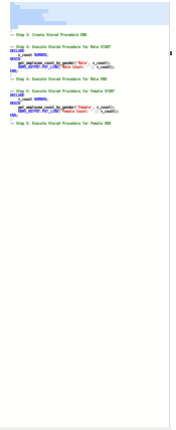
Commit complete.

Elapsed: 00:00:00.001

EXPERIMENT 8

Step 3 – Create Stored Procedure

```
20 -- Step 3: Create Stored Procedure START
21 CREATE OR REPLACE PROCEDURE get_employee_count_by_gender (
22     p_gender IN VARCHAR2,
23     p_count OUT NUMBER
24 )
25 AS
26 BEGIN
27     SELECT COUNT(*)
28     INTO p_count
29     FROM employee
30     WHERE gender = p_gender;
31 END;
32 /
```



Query result **Script output** DBMS output Explain Plan SQL history



```
SQL> CREATE OR REPLACE PROCEDURE get_employee_count_by_gender (
      p_gender IN VARCHAR2,
      p_count OUT NUMBER
    )...
Show more...
```



Procedure GET_EMPLOYEE_COUNT_BY_GENDER compiled

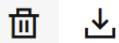
Elapsed: 00:00:00.021

EXPERIMENT 8

Step 4 – Execute Stored Procedure

```
36  -- Step 4: Execute Stored Procedure for Male START
37  DECLARE
38  |...v_count NUMBER;
39  BEGIN
40  |...get_employee_count_by_gender('Male', v_count);
41  DBMS_OUTPUT.PUT_LINE('Male Count: ' || v_count);
42  END;
43  /
```

Query result **Script output** DBMS output Explain Plan SQL history



```
SQL> DECLARE
      v_count NUMBER;
      BEGIN
      get_employee_count_by_gender('Male', v_count);...
Show more...
```

Male Count: 3

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.112

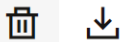
EXPERIMENT 8

Step 5 – Display Output

```
4 /      -- Step 5: Execute Stored Procedure for Female START
48 DECLARE
49     ... v_count NUMBER;
50 BEGIN
51     ... get_employee_count_by_gender('Female', v_count);
52     ... DBMS_OUTPUT.PUT_LINE('Female Count: ' || v_count);
53 END;
54 /
55      -- Step 5: Execute Stored Procedure for Female END
```

Step 5: Execute Stored Procedure for Female START
Step 5: Execute Stored Procedure for Female START
Step 5: Execute Stored Procedure for Female START
Step 5: Execute Stored Procedure for Female START
Step 5: Execute Stored Procedure for Female START

Query result **Script output** DBMS output Explain Plan SQL history



```
SQL> DECLARE
      v_count NUMBER;
      BEGIN
      get_employee_count_by_gender('Female', v_count);...
Show more...
```



Female Count: 2

PL/SQL procedure successfully completed.

Elapsed: 00:00:00.008

Learning Outcome

After completing this experiment, the learner will be able to:

- Understand procedural programming in PL/SQL.
- Implement parameterized stored procedures using IN and OUT parameters.
- Perform dynamic data retrieval based on user input.
- Use aggregate functions like COUNT() within procedures.
- Develop reusable and efficient database logic for enterprise applications used in companies like CData and Toddle.